

2.2.1

Memory Allocation

Memory allocation can be classified as either

- Contiguous
- Linked
- Indexed

Prototypical examples:

- Contiguous allocation: arrays
- Linked allocation: linked lists

Memory Allocation

Contiguous, *adj.*

Touching or connected throughout in an unbroken sequence.

Meriam Webster

Touching, in actual contact, next in space; meeting at a common boundary, bordering, adjoining.

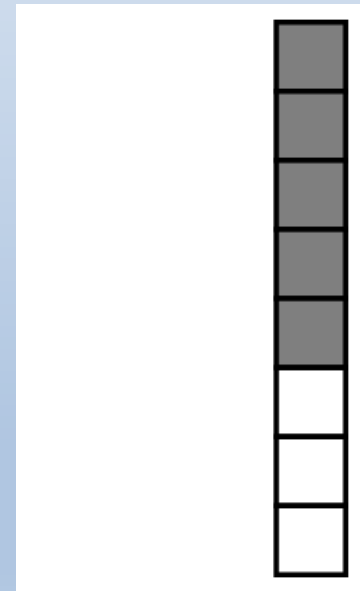
www.oed.com

2.2.1.1

Contiguous Allocation

An array stores n objects in a single contiguous space of memory
Unfortunately, if more memory is required, a request for new memory usually requires copying all information into the new memory

- In general, you cannot request for the operating system to allocate to you the next n memory locations

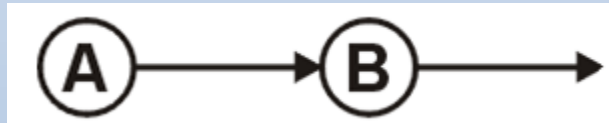


2.2.1.2

Linked Allocation

Linked storage such as a linked list associates two pieces of data with each item being stored:

- The object itself, and
- A reference to the next item
 - In C++ that reference is the address of the next node



2.2.1.2

Linked Allocation

This is a class describing such a node

```
template <typename Type>
class Node {
    private:
        Type node_value;
        Node *next_node;
    public:
        // ...
};
```



2.2.1.2

Linked Allocation

The operations on this node must include:

- Constructing a new node
- Accessing (retrieving) the value
- Accessing the next pointer

```
Node( const Type& = Type(), Node* = nullptr );  
Type value() const;  
Node *next() const;
```

Pointing to nothing has been represented as:

C	NULL
Java/C#	null
C++ (old)	0
C++ (new)	nullptr
Symbolically	Ø

2.2.1.2

Linked Allocation

For a linked list, however, we also require an object which links to the first object

The actual linked list class must store two pointers

- A head and tail:

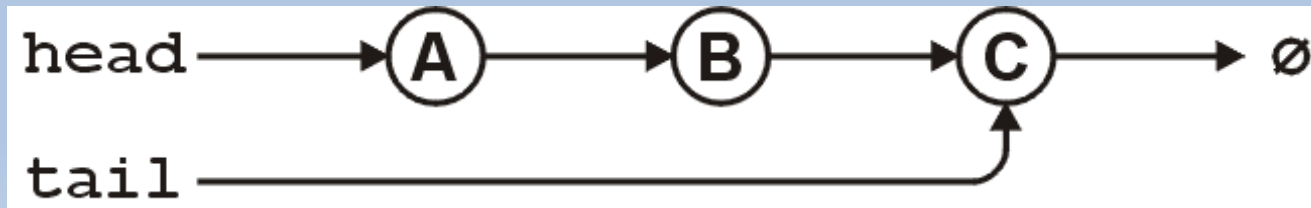
```
Node *head;
```

```
Node *tail;
```

Optionally, we can also keep a count

```
int count;
```

The next_node of the last node is assigned nullptr



2.2.1.2

Linked Allocation

The class structure would be:

```
template <typename Type>
class List {
    private:
        Node<Type> *head;
        Node<Type> *tail;
        int count;
    public:
        // constructor(s)...
        // accessor(s)...
        // mutator(s)...
};
```

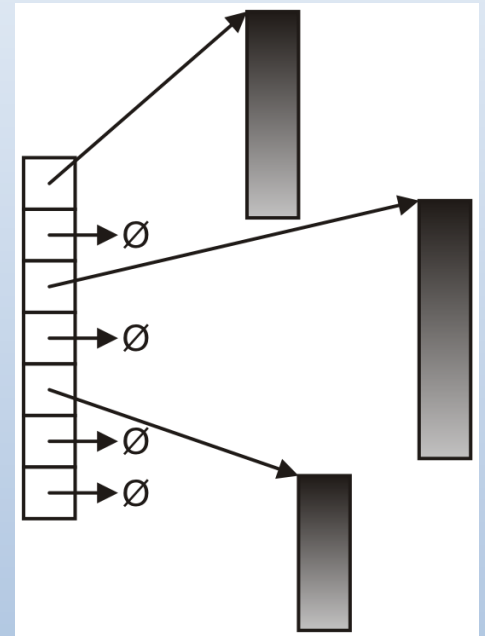
2.2.1.3

Indexed Allocation

With indexed allocation, an array of pointers (possibly NULL) link to a sequence of allocated memory locations

Used in the C++ standard template library

Computer engineering students will see indexed allocation in their operating systems course

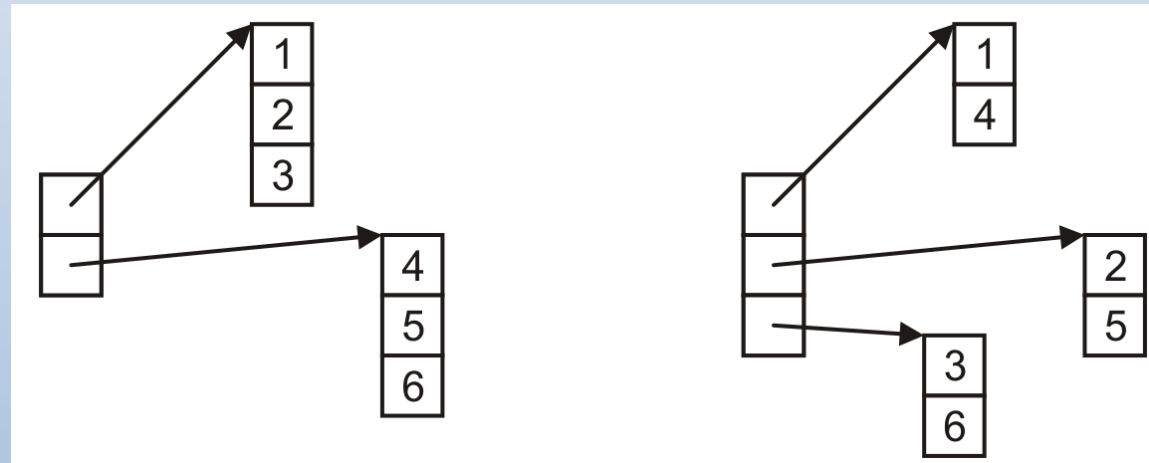


2.2.1.3

Indexed Allocation

Matrices can be implemented using indexed allocation:

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$$



2.2.1.3

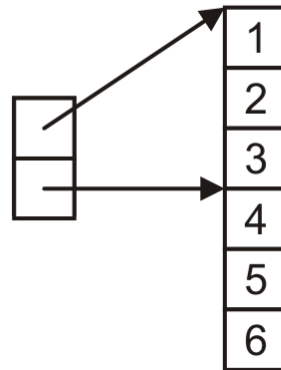
Indexed Allocation

Matrices can be implemented using indexed allocation

- Most implementations of matrices (or higher-dimensional arrays) use indices pointing into a single contiguous block of memory

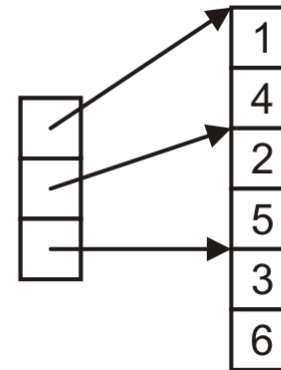
$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$$

Row-major order



C, Python

Column-major order



Matlab, Fortran

2.2.2

Other Allocation Formats

We will look at some variations or hybrids of these memory allocations including:

- Trees
- Graphs
- Deques (linked arrays)
- inodes

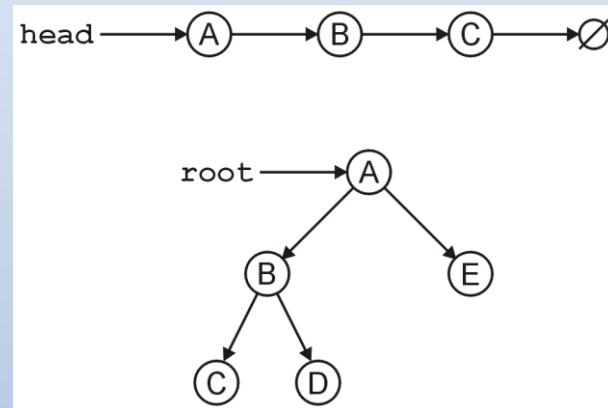
2.2.2.2

Trees

The linked list can be used to store linearly ordered data

- What if we have multiple *next* pointers?

A rooted tree (weeks 4-6) is similar to a linked list but with multiple next pointers



2.2.2.2

Trees

A tree is a variation on a linked list:

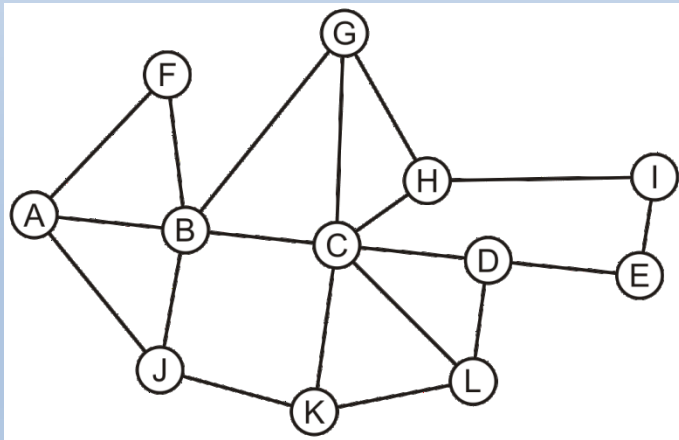
- Each node points to an arbitrary number of subsequent nodes
- Useful for storing hierarchical data
- We will see that it is also useful for storing sorted data
- Usually we will restrict ourselves to trees where each node points to at most two other nodes

2.2.2.2

Graphs

Suppose we allow arbitrary relations between any two objects in a container

- Given n objects, there are $n^2 - n$ possible relations
 - If we allow symmetry, this reduces to $\frac{n^2 - n}{2}$
- For example, consider the network

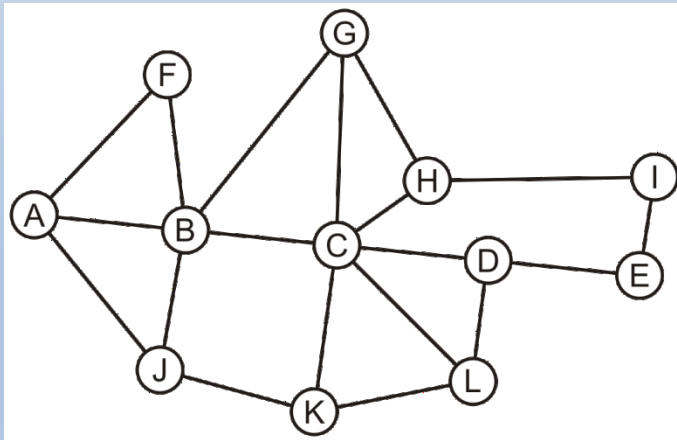


2.2.2.2

Arrays

Suppose we allow arbitrary relations between any two objects in a container

- We could represent this using a two-dimensional array
- In this case, the matrix is *symmetric*



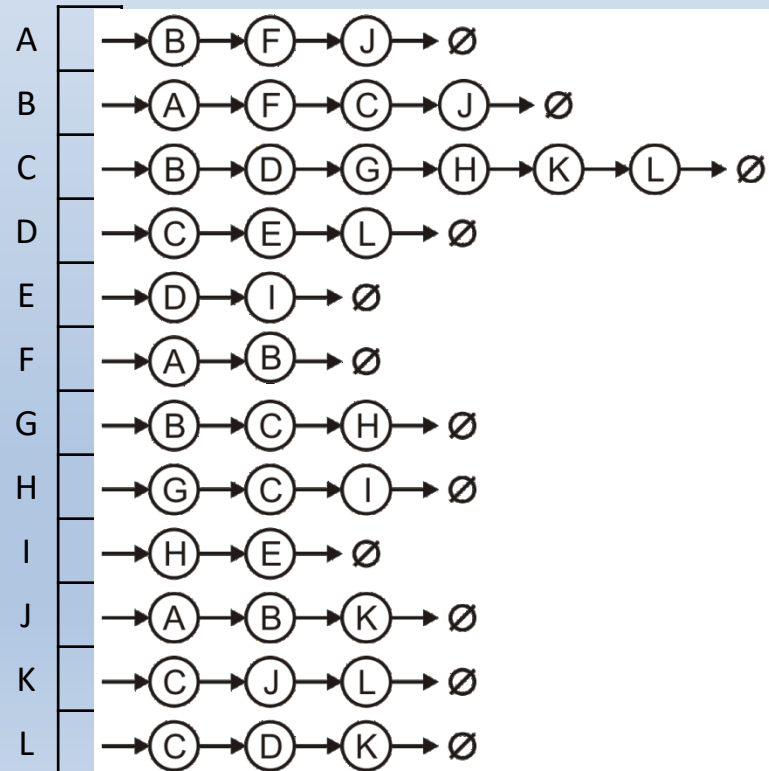
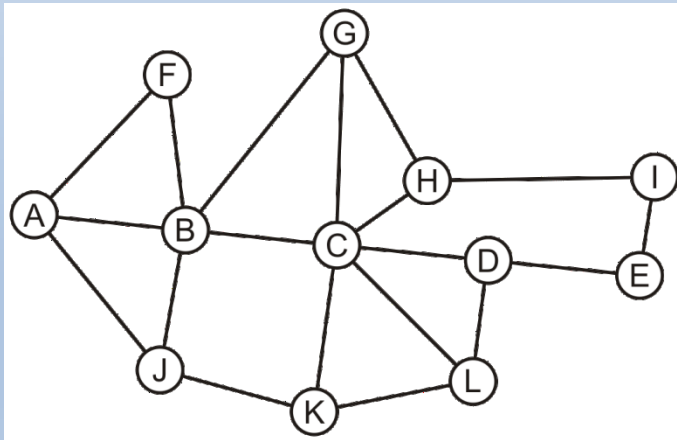
	A	B	C	D	E	F	G	H	I	J	K	L
A		x				x				x		
B	x		x			x	x			x		
C		x		x			x	x			x	x
D			x		x							x
E				x					x			
F	x	x										
G		x	x					x				
H			x				x		x			
I					x			x				
J	x	x									x	
K			x							x		x
L			x	x							x	

2.2.2.2

Array of Linked Lists

Suppose we allow arbitrary relations between any two objects in a container

- Alternatively, we could use a hybrid: an array of linked lists



2.2.2.3

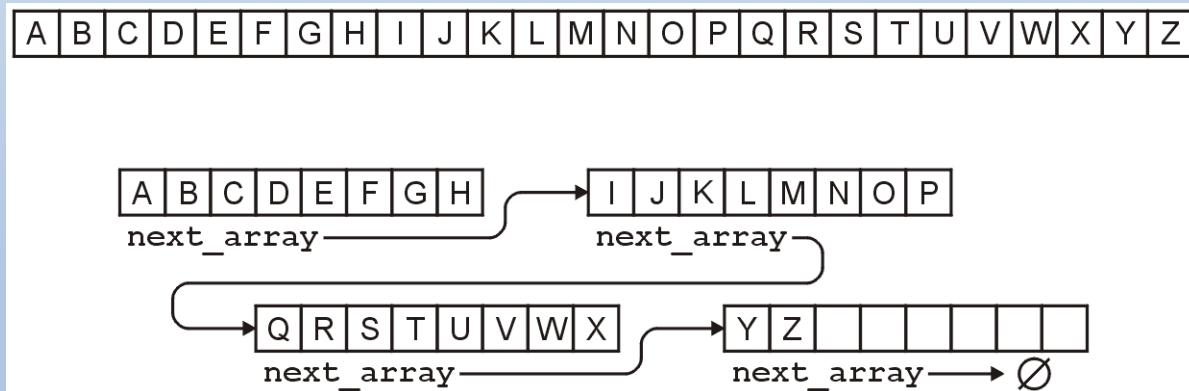
Linked Arrays

Other hybrids are linked lists of arrays

- Something like this is used for the C++ STL deque container

For example, the alphabet could be stored either as:

- An array of 26 entries, or
- A linked list of arrays of 8 entries

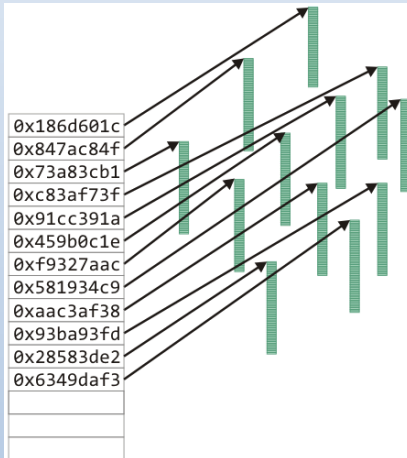


2.2.2.4

Hybrid data structures

The Unix inode was used to store information about large files

- The first twelve entries can reference the first twelve blocks (48 KiB)

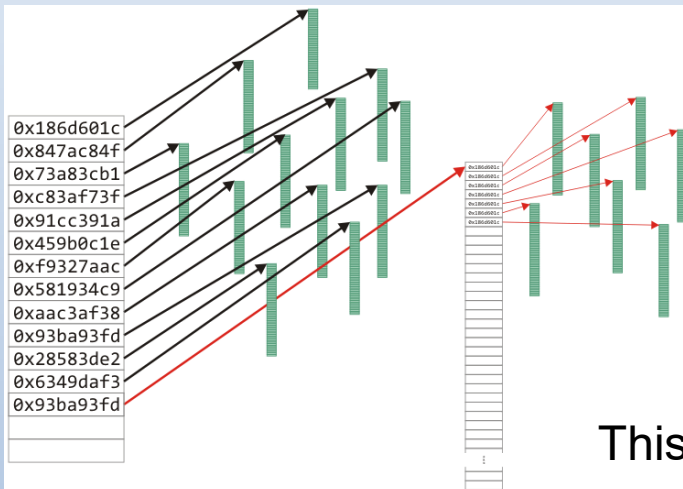


2.2.2.4

Hybrid data structures

The Unix inode was used to store information about large files

- The next entry is a pointer to an array that stores the next 1024 blocks



This stores files up to 4 MiB on a 32-bit computer

2.2.2.4

Hybrid data structures

The Unix inode was used to store information about large files

- The next entry has two levels of indirection for files up to 4 GiB

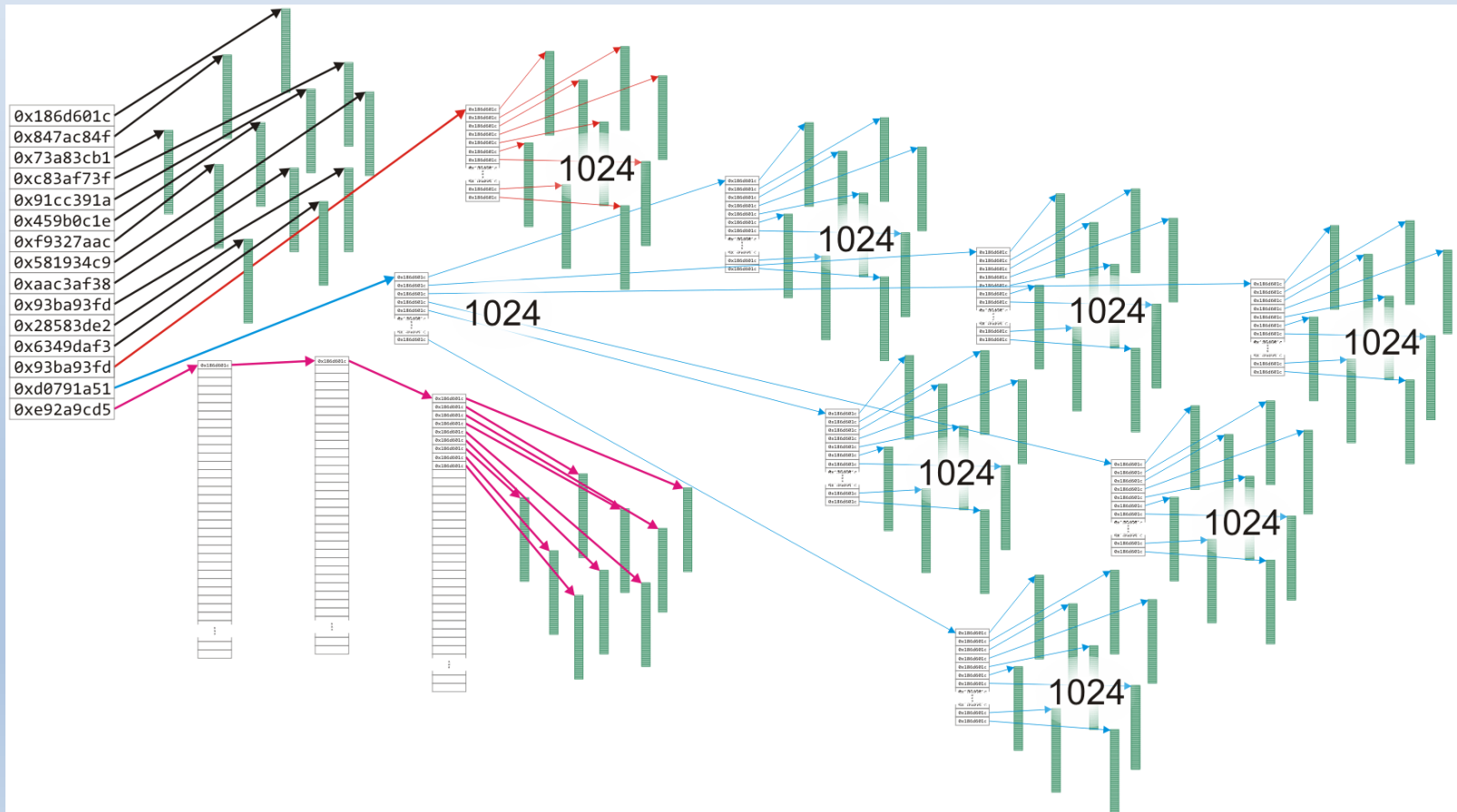


2.2.2.4

Hybrid data structures

The Unix inode was used to store information about large files

- The last entry has three levels of indirection for files up to 4 TiB



2.2.3

Algorithm run times

Once we have chosen a data structure to store both the objects and the relationships, we must implement the queries or operations as algorithms

- The Abstract Data Type will be implemented as a class
- The data structure will be defined by the member variables
- The member functions will implement the algorithms

The question is, how do we determine the efficiency of the algorithms?

2.2.3

Operations

We will use the following matrix to describe operations at the locations within the structure

	Front/1st	Arbitrary Location	Back/n^{th}
Find	?	?	?
Insert	?	?	?
Erase	?	?	?

2.2.3.1

Operations on Sorted Lists

Given an sorted array, we have the following run times:

	Front/1 st	Arbitrary Location	Back/ n^{th}
Find	Good	Okay	Good
Insert	Bad	Bad	Good* Bad
Erase	Bad	Bad	Good

* only if the array is not full

2.2.3.2

Operations on Lists

If the array is not sorted, only one operations changes:

	Front/ 1^{st}	Arbitrary Location	Back/ n^{th}
Find	Good	Bad	Good
Insert	Bad	Bad	Good* Bad
Erase	Bad	Bad	Good

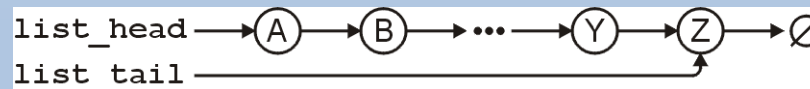
* only if the array is not full

2.2.3.3

Operations on Lists

However, for a singly linked list where we have a head and tail pointer, we have:

	Front/1 st	Arbitrary Location	Back/ n^{th}
Find	Good	Bad	Good
Insert	Good	Bad	Good
Erase	Good	Bad	Bad



2.2.3.3

Operations on Lists

If we have a pointer to the k^{th} entry, we can insert or erase at that location quite easily

	Front/ 1^{st}	Arbitrary Location	Back/ n^{th}
Find	Good	Bad	Good
Insert	Good	Good	Good
Erase	Good	Good	Bad

- `list_head` → (A) → (B) → ... → (Y) → (Z) → ∅
`list_tail` → (Z)
- Note, this requires us to modify the value stored in the k^{th} node
 - This is a common co-op interview question!

2.2.3.4

Operations on Lists

For a doubly linked list, one operation becomes more efficient:

	Front/1 st	Arbitrary Location	Back/ n^{th}
Find	Good	Bad	Good
Insert	Good	Good	Good
Erase	Good	Good	Good

